Bilkent University

Department of Computer Engineering

Senior Design Project

Final Report

Bil-Events : A mobile app to organize events in Bilkent University

Team Members

Osman Can Yıldız -21302616

Can Savcı - 21300803

Kasım Sarp Ataş - 21502006

Supervisor: Halil Altay Güvenir

Jury Members: Fazlı Can and Özgür Ulusoy

Innovation Expert: Barış Misman

Final Report
December 17, 2020

This report is submitted to the Department of Computer Engineering of Bilkent University in partial fulfillment of the requirements of the Senior Design Project course CS491/492

Table of Contents

1. Introduction **3**

2. Requirements Details **3**
2.1 Functional Requirements 3
2.2 Non-Functional Requirements 4
2.2.1 Usability 4
2.2.2 Reliability 4
2.2.3 Extensibility 4
2.2.4 Privacy 5
2.2.5 Flexibility 5
2.2.6 Portability and Accessibility 5
2.2.7 Sustainability 5

3. Final Architecture and Design Details **5**
3.1 Subsystem Decomposition 5
3.1.1 Client Subsystem 7
3.1.1.1 View Component 7
3.1.2 Server Subsystem 7
3.1.2.1 Controller Component 7
3.1.2.2 Model Component 8
3.1.2.3 Recommender System Component 8
3.2 Hardware/Software Mapping 9
3.3 Persistent Data Management 10
3.4 Access Control and Security 11

4. Development/Implementation Details **11**
4.1 Frontend 12
4.1.1 Login Page 12
4.1.2 Signup Page 13
4.1.3 Forgot Password Page 14
4.1.4 Upcoming Events Page 15
4.1.5 Selected Events Page 16
4.1.6 Recommended Events Page 17
4.1.7 Past Events Page 18
4.1.8 Club List Page 19
4.1.9 Add Event Page 20
4.1.10 Components 20
4.1.10.1 Post Component 20
4.1.10.2 Club Component 21
4.1.10.3 Header Component 21
4.1.10.4 Footer Component 22
4.1.10.4.1 Users 22
4.1.10.4.2 Club Leaders 22
4.2 Backend 22
4.2.1 models.py 24
4.2.2 serializers.py 24
4.2.3 views.py 25
4.2.4 urls.py 25

4.2.5 settings.py 26
4.3 Database 26
4.3.1 Participant 27
4.3.2 Club 27
4.3.3 Event 27
4.3.4 Recommended Event 28
4.4 Recommender Systems 29
4.3.1 Data Sparsity 30
4.3.2 Cold Start Problem 31
4.3.3 First Rater Problem 31
4.3.4 Troller Rater Problem 31

5. Testing Details 33

6. Maintenance Plan and Details 34

7. Other Project Elements 35
7.1 Consideration of Various Factors in Engineering Design 35
7.1.1 Adaptation to Bilkent University and Other Universities 35
7.1.1.1 Safety 35
7.1.1.2. Economic 35
7.1.1.3. Global 35
7.1.2 Adapting to Pandemic 36
7.1.2.1 Safety 36
7.1.2.2 Social 36
7.2 Ethics and Professional Responsibilities 37
7.2.1 Ethical Issues 37
7.2.2 Professional Issues 37
7.3 Judgements and Impacts to Various Contexts 38
7.4 Teamwork Details 39
7.4.1 Contributing and functioning effectively on the team 39
7.4.2 Helping Creating A Collaborative and Inclusive Environment 39
7.4.3 Taking Lead Role and Sharing Leadership On The Team 39
7.4.4 Meeting Objectives 40
7.5 New Knowledge Acquired and Applied 40
7.5.1 Android Programming with React Native Framework 40
7.5.2 Recommendation Engine 40
7.5.3 Visual Studio Code 41
7.5.4 Heroku 41
7.5.5 Django Rest Framework 41
7.5.6 Server Handling 42
7.5.7 Adobe Photoshop CC 42

8. Conclusion and Future Work 42

9. Glossary 43

10. References 44

1. Introduction

According to YÖK's website, there are currently over 13.000 students studying in Bilkent. Most of these students are the members of some student clubs and these clubs host many different events for Bilkenters to warm up to the university and to socialize. However, there are several problems with the event notification system. First of all, since there is no unified notification system that people can access the information about events from various places. There are three main ways to access event information. One of them is from webmail notifications (BAIS), another one is from students clubs' posters and the third one is from SRS' GE 250/251 notifications. However, all of these three have their own flaws. To clarify what these flaws are actually, we should examine the event notification systems. The first one, the BAIS mail system, which sends over ten mail per day about not only events but also all notifications about Bilkent University and the seminars. It is hard to keep track of events and remember their dates. In addition to this, the BAIS mail system sends the emails unaesthetically, it causes resentment even for the students who want to invest their valuable time by attending the events. The second one, using paper posters is outdated, also a huge waste of paper and they are not well-distributed. They are so randomly distributed that it causes distraction among the students who want to find the right event for themselves. For instance, you can't find specific events posters in every building as they are not hung everywhere or they are overlapped when they are hung. For the students in the East Campus, this issue cannot even be mentioned because they cannot reach information about the events, because the students don't have any idea how to find the locations of the events and the events are generally held in the Main Campus, the distance between East Campus and the location of the events and lack of knowledge on how to find the location give resentment to the students who are in East Campus. BAIS e-mails and posters are not descriptive enough to handle that problem we mentioned. The last point we want to mention is that the update on SRS' event notification system does not even show the events that don't give GE 250/251 points. As a group of students who are in this system, we observed and suffered from these problems and as a solution we offer an app which will be accessed by everyone.

2. Requirements Details

2.1 Functional Requirements

- Users can login the application with their personal ID number and password.
- Users can create accounts with their student ID's.
- Forgotten password is sent to the "Forgot Password" page.
- The user sees details such as name, location, starting time, capacity and GE 250/251 Points.

- The users can search for the events in Bilkent.
- The user can choose to attend an event.
- Users can join the Zoom channel of the event by clicking the event's Zoom button.
- Chosen events will be seen in the selected event page.
- In selected events, There is a button that shows the user to leave the event in.
- Bil-Event will list events in a way that specific events which are recommended events have their own segment. According to users selected events tags, recommended events will be suggested to users. Recommended events are special to each user.
- Selected past events can be seen in the past events page and the user can rate the event here.
- The user can choose to create an event but to create an event, the user must be the official club leader.
- There is a button that shows the user to join the event in.
- Users can see club lists with its leader, description and tags.
- After the event is posted on the app, everyone can access it immediately.
- Navigation between pages is provided.

2.2 Non-Functional Requirements

2.2.1 Usability

The application will be easy to use for both organizers, who are the club leaders, and participants. Therefore, the user interface must be simple enough for both users. UI/UX Design are taken into account when we develop our application.

2.2.2 Reliability

As events in Bilkent University will be added by club leaders, the events are expected to happen. As the seminar is managed by the collaboration between student club leaders and Bilkent University, events will be reliable. The rating of events and the reviews about the events are expected to increase the reliability of the events.

2.2.3 Extensibility

The application will be easy to add features (e.g. idea of chat, friend requests etc). Hence, it should be easy to update with respect to both its background implementation and user interface design. The requirements of the users are going to be taken into account when adding new features.

2.2.4 Privacy

User information will be private. Unless taking permission of the user, user information should not be accessed by anyone. The collected data from users are encrypted and protected with the backup and encryption keys. Selling the user data to the third parties is prohibited regardless of the consent of users.

2.2.5 Flexibility

The application will be suitable for all Bilkent students, even preparatory students. It should also provide online features regarding the different user expectations.

2.2.6 Portability and Accessibility

As the product is a mobile application, it will be portable and accessible for all Bilkent students from everywhere. Internet connection is needed. The app won't require high quality hardware. We could consider adding some accessibility features to our disabled users.

2.2.7 Sustainability

Since the highschool students are applying to Bilkent University every year and students are actively participating in the events of Student Clubs because of their interest in the club or GE 250/251, the application preserves its sustainability.

3. Final Architecture and Design Details

3.1 Subsystem Decomposition

Our application works in both the Android and iOS smartphones. Our project follows the Client-Server architecture and MVC pattern. We have two main subsystems named as Client and Server.

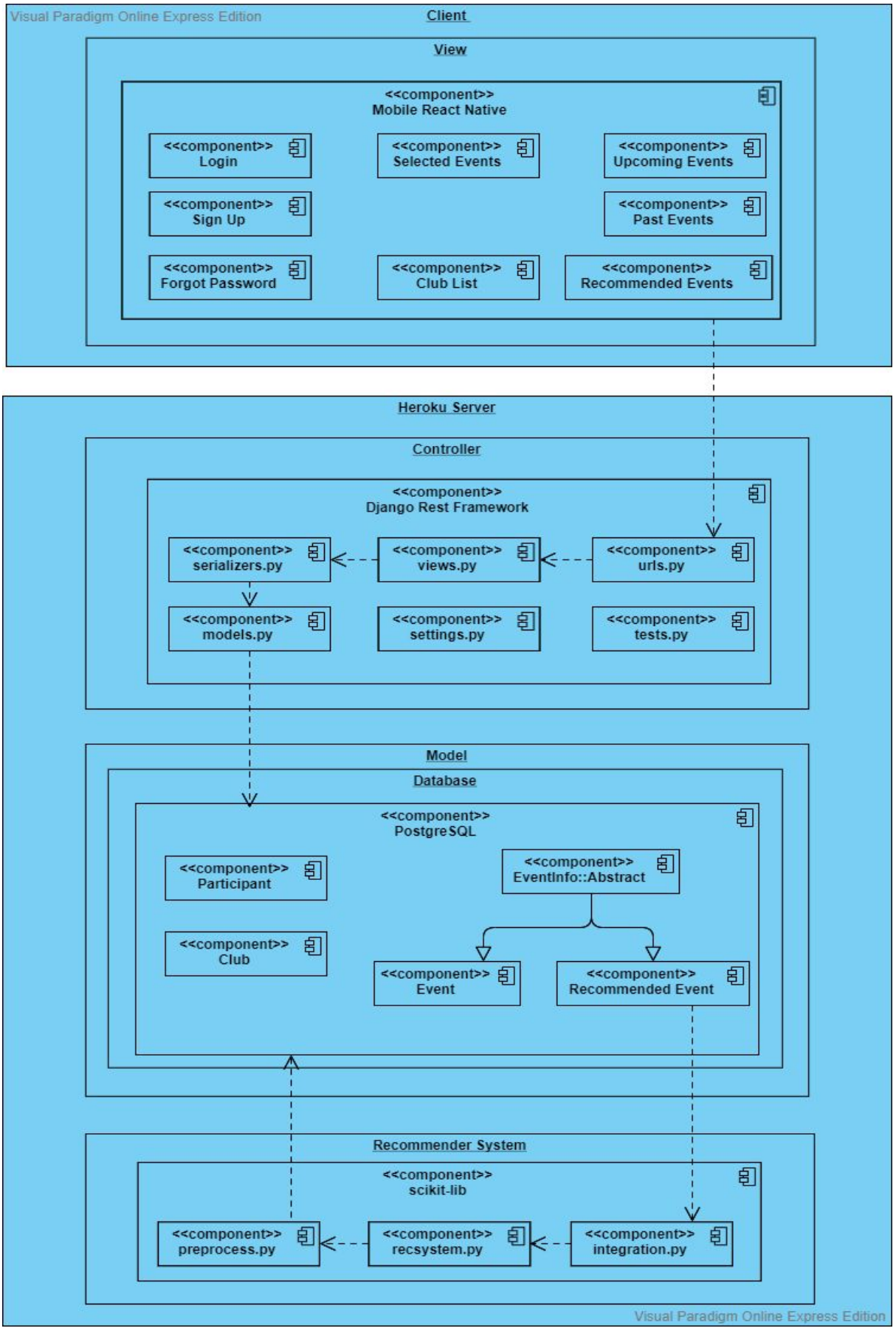

Figure 1 : Subsystem Decomposition

3.1.1 Client Subsystem

In the Client Side architecture, we have a main component named as View.

3.1.1.1 View Component

The View component is implemented by React Native framework which is a React-based JavaScript framework and it helps to create the native applications. The View component consists of all of the UI / UX components such as Login, Sign Up, Forgot Password, Selected Events, Upcoming Events, Past Events and Recommended Events pages, the components of the pages mentioned above and the user interactions of the pages. The Client Side visualizes the information which is coming from the Server side via View component and sends the user interactions to the Server Side as information.

3.1.2 Server Subsystem

In the Server Side, we have three main components named as Controller, Model and Recommender System.

3.1.2.1 Controller Component

In the Controller component, we execute the CRUD operations of the Model component. The Controller component can read the data and send it to the View component or it can execute the create, update and delete operations and send the changes to the Model component. The Controller component is responsible for managing the data validation, data order, data representation authorization and authentication as well. The data validation consists of checking data length, data type and custom data features which shifts from the application to the application. For instance, in the Controller component and in the serializers.py module, we perform the check of the domain name of the Participant email and we expect the domain of the email to have a BCC domain. In the data order, the Controller component manages the order and serialization of the data. The Controller component can sort the data in the database table with respect to the specified key and it manages which fields are going to be serialized, how they are serialized and exported. It manages the authentication and authorization by specifying who is going to access, how the user is going to access and which portion of the data the user is going to access. In addition to this, the Controller component manages the export of data as the JSON format. Specifically, it controls in which endpoint the data is exported and which data is exported as the JSON format. It controls which operations are going to be executed on the data on which endpoint as well. It controls which endpoints are going to be used and in which settings they are going to operate such as which endpoints are accessible to the end user or which endpoints are not accessible to them. In the settings, the Controller component manages the settings of the application such as which database engine will be used to store the data coming from the user, installed apps, middlewares, password validators and static and media files stored in the database.

3.1.2.2 Model Component

In the Model component, we store the database models. Database models are the data tables we hold in our database, which is managed by our Controller component and viewed by our View component. We used PostgreSQL database engine to store our Models in the Model component. Our models are Participant, Club, Event, Recommended Event and EventInfo. The EventInfo model is an abstract model which can be inherited by Event and Recommended Event models. The Participant model represents the participants of the events, the Club model represents the clubs, the Event model represents the events, the EventInfo model represents the general events and the Recommended Events model represents the events which are going to be recommended to the participants in the Bilkent University.

3.1.2.3 Recommender System Component

In the Recommender System component, the recommendation engine operates. The engine first takes the data from the database in the model component, then preprocesses the data. The preprocess of the data consists of the steps of importing the essential libraries such as Numpy and Pandas libraries, fetching the data from the database, examining the data such as checking the missing or unexplainable atomic values, inferring the categorical values from the raw data, displaying, plotting and performing the statistical operations on the preprocessed data, splitting the data into training, validation and test set and performing the feature scaling and normalization operations on the data to speed up the process of the algorithm. After preprocessing data is finished, we apply several learning algorithms to the data to find the best model which can minimize the error. When the best model is chosen, it applies the best learning model to the preprocessed data. After the results are generated, the Recommender System component exports the data to the Recommended Event model in the database[11].

In the preprocess.py, we fetch the data from the database. Then, we perform the data preprocess operations mentioned above. The operations mainly consist of Information Retrieval and Data Mining operations. For instance, we use events score, events current capacity and events maximum capacity data as it is; however, we infer the participant's department from his/her email or the participant's applicant day to the Bilkent University from his/her Bilkent ID. After the Information Retrieval and Data Mining operations, we aggregate the results of these operations, create a data frame using Pandas library and send it to the recevents.py module.

In the recevents.py, the application of the learning algorithm is performed. The learning algorithm is built by using the Scikit library in the Python programming language. The reason why the libraries are used is that our recommendation engine should generate Just-In-Time (JIT) results and export them to

the database. It increases the efficiency of our application as well. That is why we tried to build our recommendation engine as an efficient part of the service.

In the integration.py, actual integration is performed. First, we import the upcoming events from the Events table to the Recommended Events table. After the processes are executed in the recevents.py, we export the results to the Recommended Events table in the database. Following the processes mentioned above, the user can see, join and leave the events and at the same time the user can see the portion of the events in the Recommended Events page in the View component.

3.2 Hardware/Software Mapping

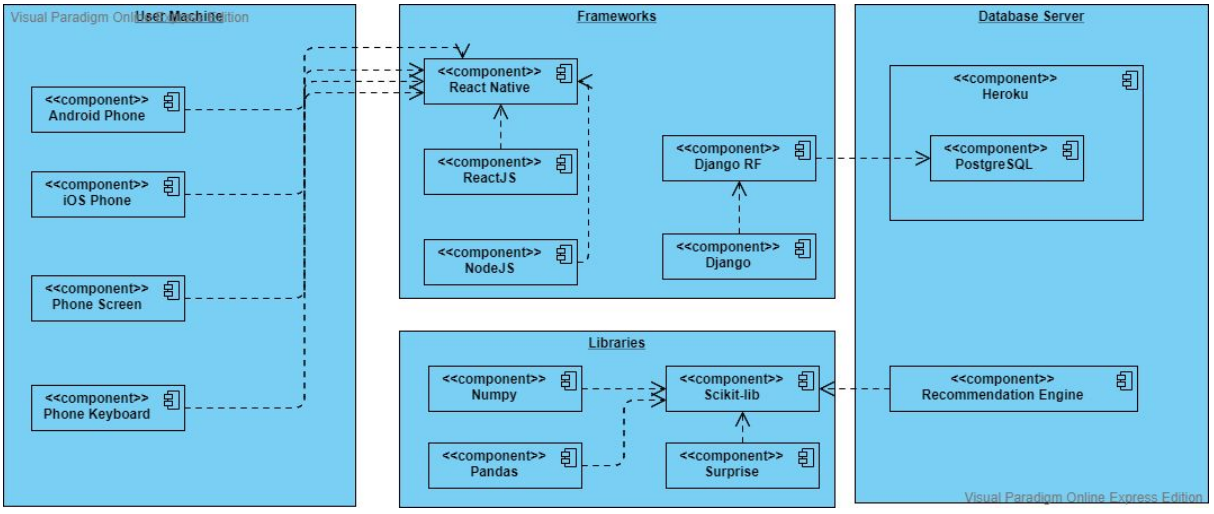

Figure 2 : Hardware/Software Mapping

Bil-Events initially runs on the phones which have an Android or iOS operating system. The application uses the phone screen and phone keyboard for the I/O operations. Any kinds of user interactions will be stored on the Heroku server with PostgreSQL database management system.

Thus, it requires internet connection, a stable screen and a stable OS. Client-server architecture is used to interact with the user. Client-side communicates with the end-users via UI and UX helps them to use the application effectively. Data management operations are handled on the server-side of the application. Client-side needs to communicate with the server side and perform CRUD operations such that the user interactions can be stored in the dataset. This communication will be provided by HTTP which is an asynchronous request-response protocol. An HTTP client initiates a request by establishing a Transmission Control Protocol (TCP) connection to a particular port on a server and it is provided to configure the settings in the server side. An HTTP server listening on that port waits for a client's request message. Upon receiving the request, the server sends back a status code along with the response data, such as HTTP/1.1 200 OK and a message of its own[12]. The body of this message

is typically the requested resource, sometimes an error message or other information will be returned to the end user such to warn the user and send a better request.

To ensure the client-server side communication, the frameworks are used. The React Native framework is used to capture the user interactions and send it to the server side. React Native framework uses two main frameworks as an infrastructure. One of them is ReactJS which is a popular JavaScript framework. It is a frontend framework which helps a developer to create an interactive UI and provide a better UX. The other one is NodeJS framework which is a backend framework and it helps to perform the CRUD operations on the server side. The main purpose of React Native is to build the native applications which can run in the smartphones and browsers no matter what the features of the smartphones and browsers are. On the server side, we used the Django Rest Framework which serializes the data in the database via widely used JSON format. The Django Rest Framework uses the Django Framework as an infrastructure. The Django framework is a backend framework and it helps to perform the CRUD operations on the server side without using the SQL Syntax. The Django Rest Framework which performs the CRUD operations on the data and serializes the data sends the serialized data to the client side of the application. Since the JSON format is a very widely used format in the JavaScript applications, the client side can obtain the serialized data to display it to the end users.

The server side manipulates the data in the PostgreSQL database engine which is an open source-relational database management system. PostgreSQL features transactions with Atomicity, Consistency, Isolation, Durability (ACID) properties, automatically updatable views, materialized views, triggers, foreign keys, and stored procedures[13]. Finally, the data in the PostgreSQL will be transported to the Recommendation Engine which uses the essential libraries for data management, data processing and the application of the learning algorithm. Pandas library is generally used for data manipulation and creating a dataset to apply the learning algorithm. Numpy, Scikit-lib and Surprise libraries are used for application and improvement of the learning algorithm. After the execution of the modules in the Recommended Engine, the database will be updated and the results of the Recommended Events will be updated and displayed in the client-side to the end users.

3.3 Persistent Data Management

Bil-Events stores various information about the Participants, the Clubs and the Events such as Bilkent ID, club name, event name, club members and event participants. The data management requires a lot of infrastructure to be used in the Recommendation Engine or to be displayed in the client side. For this purpose, we will store the attributes in the database and PostgreSQL database engine is used for this operation. We mentioned the advantages of the PostgreSQL above and that is why it is a very

widely used database engine. In the Heroku server, an online PostgreSQL database is provided and the database is well maintained. The data is ported across the client and the server side of the application and the data requires access at fine levels of detail by multiple users as well; therefore, we needed to store the data in the database. The downside of the server is that it cannot store the dynamic media files such as the user uploaded images. However, the database on Heroku can store the static files; therefore, we uploaded the club logo images to the server to use them in the application.

3.4 Access Control and Security

The Bil-Events application requires the user to sign-up in the first place. The user must have a valid Bilkent ID and a valid email which has the BCC domain. It prevents the spam/bot account because we can send the JSON serializable data to BCC and they can check if there is one or more invalid accounts or not. In addition to this, we check the credibility of the password. The password should include at least one uppercase character, one lowercase character, one numeric character and one symbol. Using the checker mechanism, we ensure the security of the user accounts by enforcing them to create the stronger passwords.

In order to prevent malicious club and event addings from the user, we took the necessary precautions. Any user but admin cannot add a new club to the database. The club leaders have to send the detailed information about their clubs, after the necessary bureaucratic processes are done such as informing the Student Activity Center in Bilkent University, the clubs will be added to the database[14]. For the events, only the club leaders can add them. A club leader cannot add an event of another club. Taking these precautions, we want to ensure the maximum user experience for every member of the Bilkent University. In addition to this, the participants cannot see the detailed information of other participants; thus, we ensured the privacy of each member in the Bilkent University.

4. Development/Implementation Details

Our project simply consists of several routes and several functionalities. These functions and routes can be used by the users. A database from the server is needed to keep our records and a server is needed to meet the functionality. React Native which is used in frontend cannot access the database by default since there is a security issue which is caused due to the fact that database information must be kept on the device for access. To handle that problem, a function called “fetch” is used to access the data from API. With this method, we can send a CRUD request to a remote server, Heroku, and get a response from it. Our remote server communicates with our frontend, backend and database. In this way, our information is safe because there are three admins that can see the server and database and admins are our team members. In addition to this, our Recommender Systems functionality

communicates with the database and the backend whose data and models are fetched and the appropriate methods are used to preprocess the data. After that the system uses the preprocessed data as a training set and it uses the upcoming events as a test set. Our remote server ensures that the Machine Learning model is up and running smoothly.

4.1 Frontend

We chose React Native for frontend because React Native uses Javascript and Mobile-native language for its UI elements and it is really fast while using the application and provides useful libraries. Although It is a new technology which was published in 2015, its features are very easy to implement what we imagine in our mind.

To test React Native, we used Expo which is an open-source platform. In the implementation phase, by using Expo, the changes that we have done is seen on the mobile phone at the same time.

4.1.1 Login Page

When the app is opened, the user interacts with the login page. If the user has an account in Bil-Events database, that user can login to the app with his/her unique ID number and password. ID number is restricted with 8 digits as Bilkent University IDs have 8 digits. After the user has entered the ID number and password then click on login, he/she is redirected to the upcoming events page. If the user has no account, “Signup” button is provided.

Figure 3: Login Page View

4.1.2 Signup Page

In the login page, if the user clicks on the “Signup” button, the user is redirected to signup page. User enters a unique ID number and e-mail address, name,surname and password then clicks on the “Register” Button. If all input datas are correct, the user can create an account and the user is redirected to the login page. On the other hand, the user is warned about incorrect input.

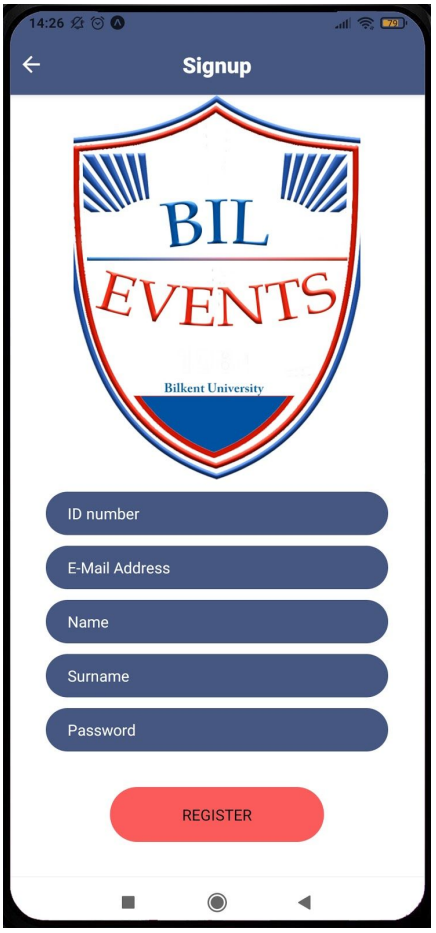

Figure 4: Signup Page View

4.1.3 Forgot Password Page

In the login page, if the user clicks on the "Forgot Password?", the user is redirected to the forgot password page. User enters the email and clicks on the "Send Email" button. A link is sent to the user about resetting the password.

Figure 5: Forgot Password View

4.1.4 Upcoming Events Page

This page comes up after logged in. In this page, upcoming events are listed. Post Components ,in 4.1.10.1, which are the posted events, are called and listed to the user.

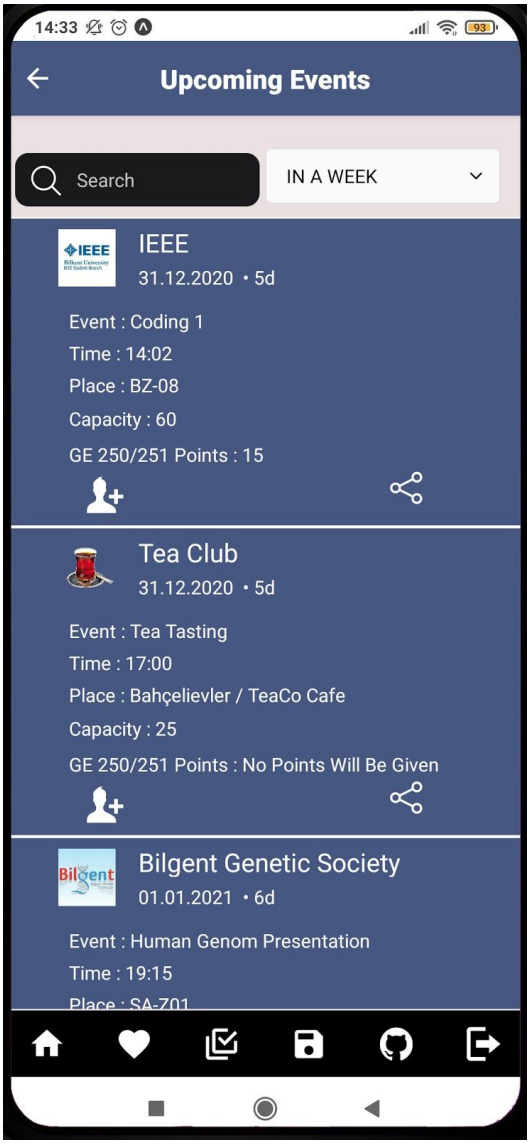

Figure 6: Upcoming Events View

4.1.5 Selected Events Page

In this page, selected events from the upcoming events page are shown to the user. The user can click on the “ x ” button with a red mark and leave from the event.

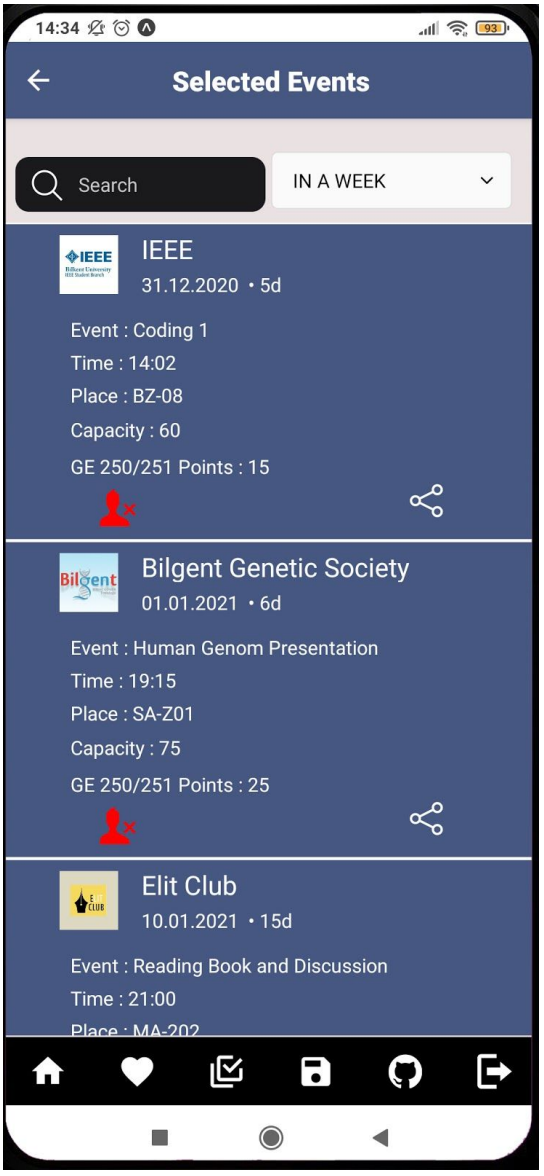

Figure 7: Selected Events View

4.1.6 Recommended Events Page

In this page, recommended events are guessed with a recommendation algorithm and demonstrated to the user. Event tags help the algorithm so that the user can see similar events which is recommended.

Figure 8: Recommended Events View

4.1.7 Past Events Page

In this page, all selected past events are listed. If the date is over, the event is deleted from the selected event page and added to the selected past events page. Users can rate the event here.

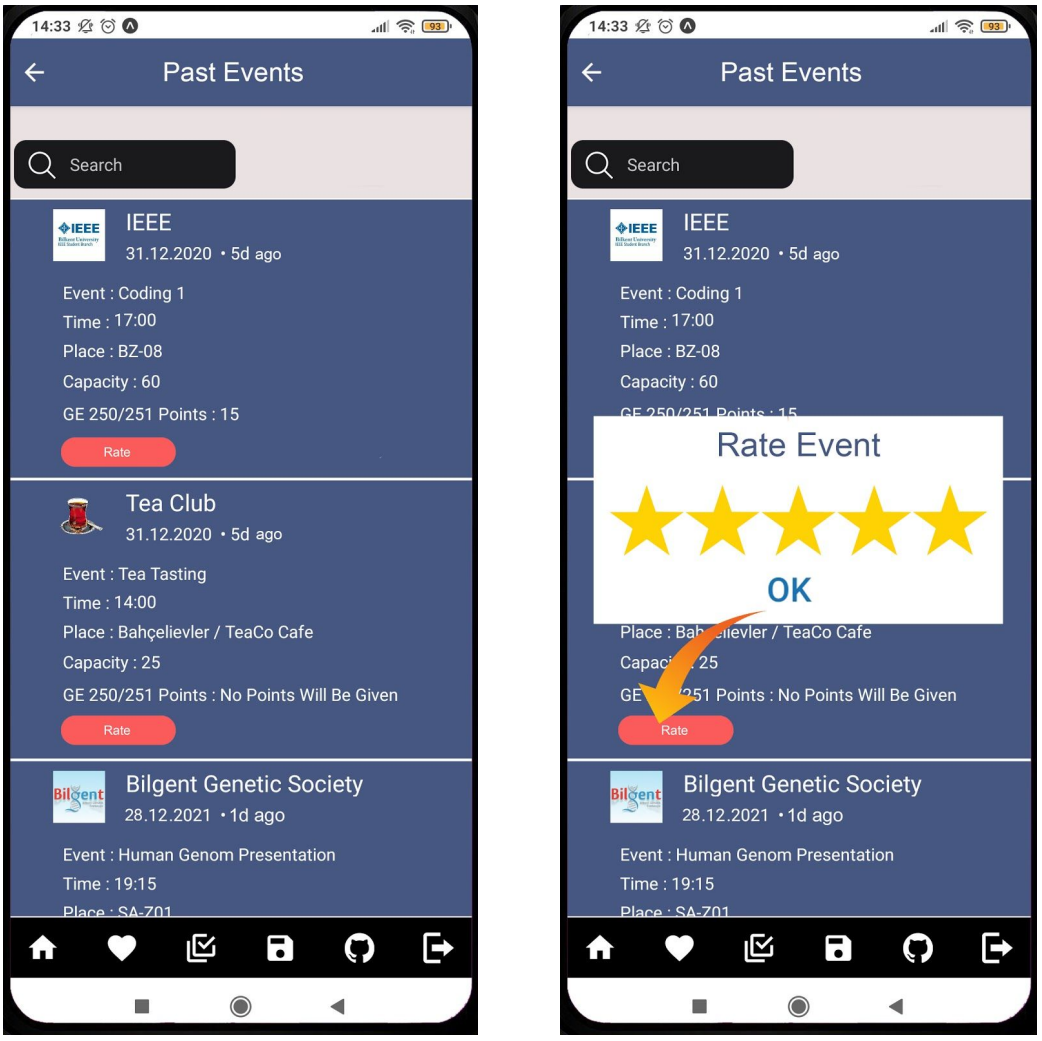

Figure 9: Past Events Views

4.1.8 Club List Page

In this page, all clubs are listed. Club leaders, club definitions and tags are demonstrated. By looking at that page, the user can search the event about a specific club on the upcoming page.

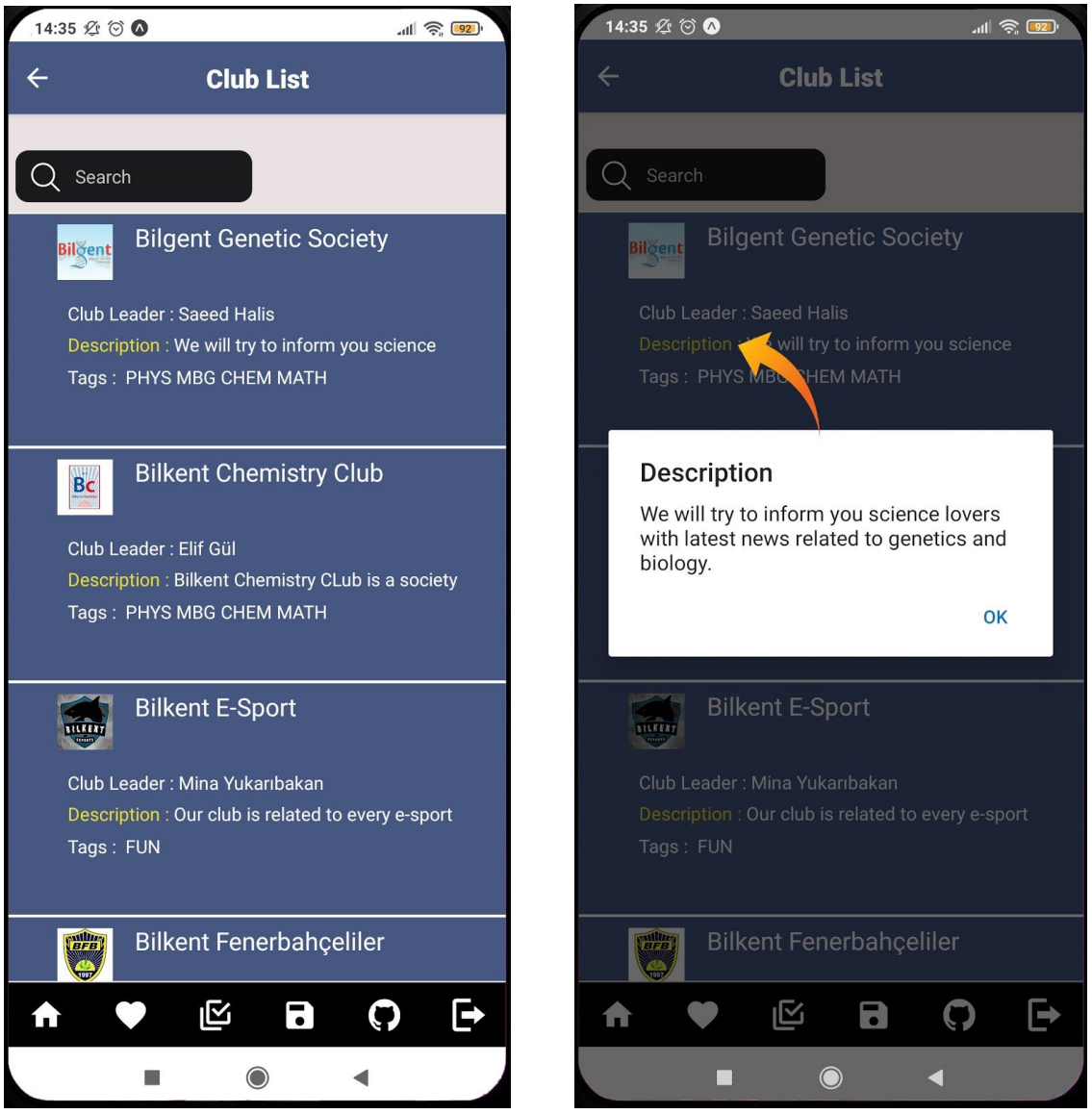

Figure 10: Club List Views

4.1.9 Add Event Page

This page is available for only club leaders. The club leader can add an event on this page. GE point is initially 0. If a user doesn't enter the GE point, “No points will be given” warning will be seen on posted events.

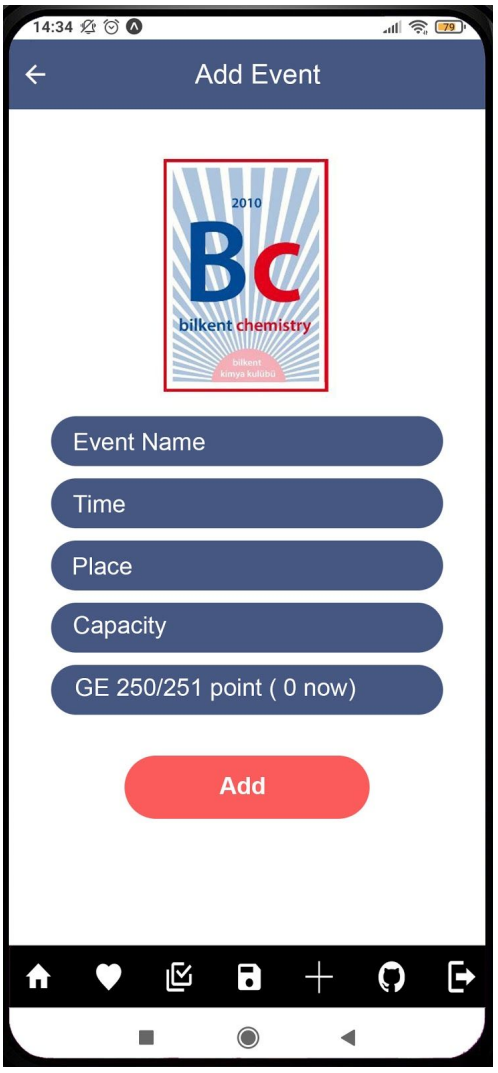

Figure 11: Add Event View

4.1.10 Components

4.1.10.1 Post Component

This event template is filled according to the events on the database. Post components are called in the upcoming events page for each event, called in selected events for each selected event and called in recommended events for each recommended event. Users can click on the “+” button which is used to select that event. After selection, the event is also seen in the selected events page,in 4.1.5. On the bottom right corner, there is a zoom link for online events.

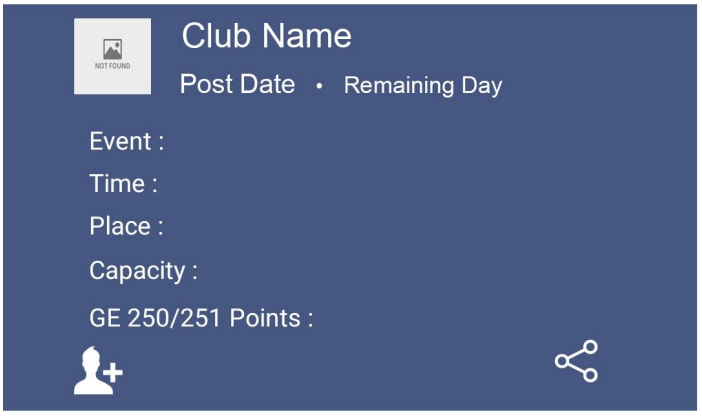

Figure 12: Post Component View

4.1.10.2 Club Component

This club template is filled according to the clubs on the database. Club components are called in the club list page for each club.

Figure 13: Club Component View

4.1.10.3 Header Component

The user can search the event by writing the event name to the search box. By changing the date limit as “In A Week”, “In A Month” or “In A Semester” Upcoming events are listed according to week, month or semester.

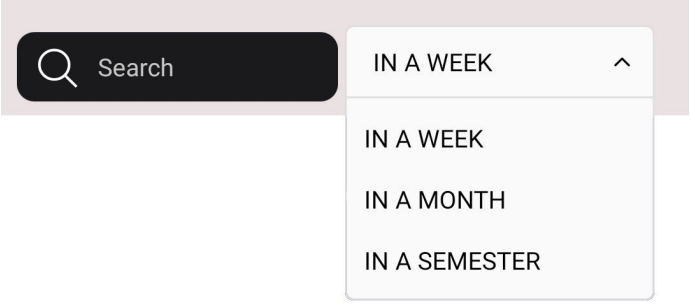

Figure 14: Header Component View

4.1.10.4 Footer Component

4.1.10.4.1 Users

This footer is available to non-leader users. Respectively, there are upcoming events, selected events, recommended events, club lists and log out icons. By clicking these icons, the user can navigate from one page to another.

Figure 15: User Footer Component View

4.1.10.4.2 Club Leaders

This footer is available to the club leader only. Respectively, there are upcoming events, selected events, recommended events, club lists, add events and log out icons. By clicking these icons, the user can navigate from one page to another.

Figure 16: Club Leader Footer View

4.2 Backend

We chose Django Rest Framework for the backend framework for several reasons. First, we needed to ensure that the data which is stored in the database would be serializable. React Native is a framework which is built in the JavaScript substratum; therefore, JSON would be reasonable to serialize the data and send to the client. In addition to this, we needed to implement a Machine Learning model which can be implemented in Python easily. However, our remote server Heroku does not support more than one programming language on the application. We evaluated the advantages and disadvantages of mainly used programming languages and we decided that Python would be a best choice since we could implement our backend and Machine Learning model with a common programming language and upload to our remote server without any problems. One of the challenging parts of our choice is that Python is generally slow compared to the other programming languages. In the worst case, 50.000 students and alumni would potentially use our end product; therefore, using the application should be fast enough to meet their needs. However, Django which is an open source framework is a web framework which is fast enough in terms of developing an application and making a query. It has lots of prebuilt libraries and features which help the optimization problems on the application and at the same time it makes developers' tasks much easier compared to the other frameworks. It is highly scalable, 50.000 people can access and benefit from our application at the same time. It prevents

Bil-Events app from the security attacks such as SQL Injection, XSS etc. and it also provides authentication and authorization features to maintain security and durability. It also has the admin panel which helps the developers or database administrators to control every aspect of the database such as authorization, authentication, CRUD operations on the database from the admin panel with a UI. In addition to this, Django provides an Object Oriented Structure to develop the database model fast and it maximizes readability when providing the structure.

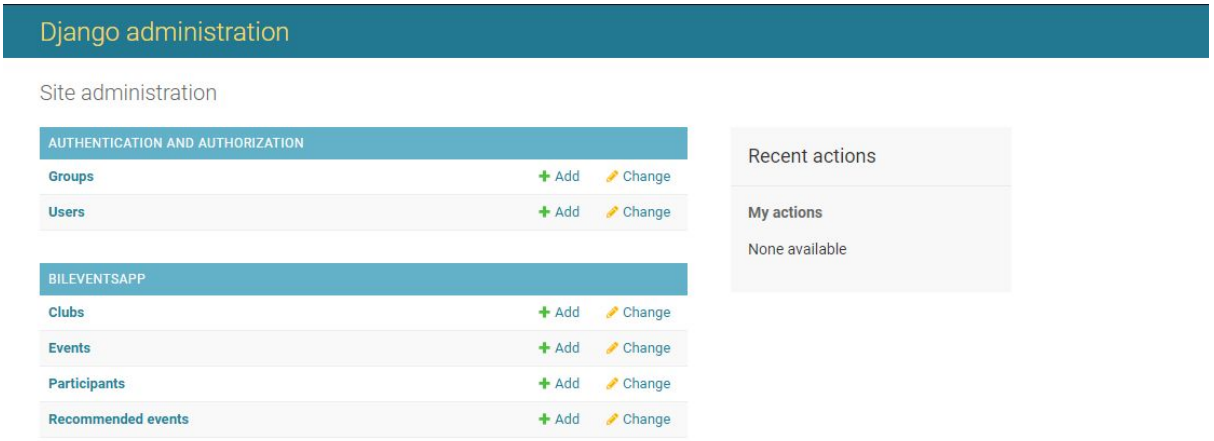

Figure 17. Django Admin Panel

Another challenging part of the project is that even though Django supports the serialization in the JSON format, the serialization and deserialization processes are done by the developers and they are not optimized enough. The second issue of using Django is that it does not provide an API endpoint structure which our client fetches the data from. Django is a powerful web framework; however, it is not much powerful in the native applications because of unavailability to create the endpoints and costly serialization procedures. Thus, we decided to use Django REST Framework with the conclusion of our extensive research. The main objective of DRF is to serialize the data in the database using JSON and create one or more endpoints so that our native application can fetch the data and use it in the client side of the application. DRF provides a separate admin panel which a database administrator can do CRUD operations in the admin panel. The most advantageous feature of DRF is that DRF supports all of the features of Django; therefore, we could combine two frameworks and design and create our backend structure with ease.

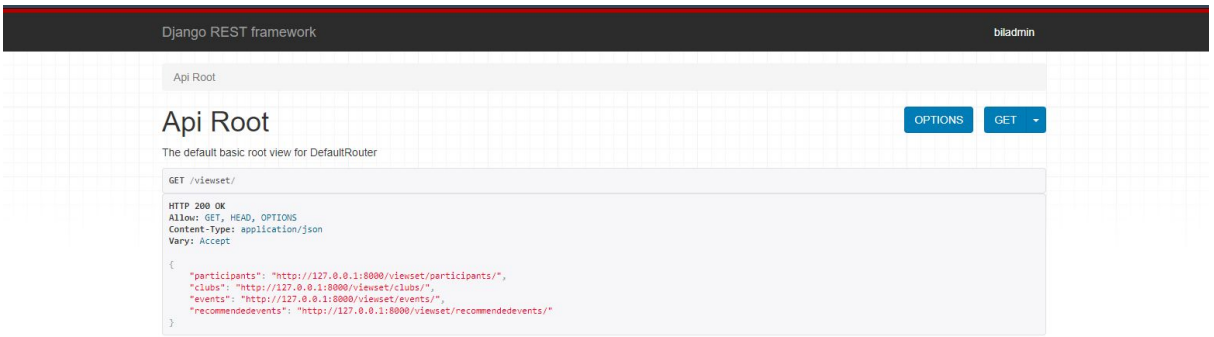

Figure 18. DRF Admin Panel

DRF has the 5 essential components which help the developers to create the backend structure of the application. After creating a virtual environment for our Python (3.5, 3.6, 3.7, 3.8, 3.9) backend and installing and running the appropriate dependencies such as Django (2.2, 3.0, 3.1) which is a dependency for DRF, DRF itself, Pillow for uploading images on the database etc. on the backend, creating a Django project and setting up the administration such as creating a superuser we started to develop our structure using 5 main components.

4.2.1 models.py

In this module, we created our database model in an Object Oriented way. We discussed and designed a model with 4 main tables which are Participant, Club, Event and Recommended Event with the specified fields in the next section. We specified the constraints of the fields such as uniqueness, primary/foreign key specification, the types of the field and so on. After that we specified the order of the tuples with respect to the predetermined attributes for each table. We also determined the relations between entities and created the relationship between entities using prebuilt OneToOneField and ManyToManyField methods. The database model will be explained in detail in the section 4.3.

4.2.2 serializers.py

In this module, we serialized the created database model in the JSON format. Since Python supports the associative array structure called Dictionary and the structure resembles the JSON format, we can propose that the Dictionary data type can be serialized and converted to the JSON format. We want to serialize our database model so that we can export to the API endpoint that we specify and our client side can fetch the data from there in a JSON format. We have written the serializer classes which are CurrentParticipantSerializer, ClubSerializer, EventSerializer and RecommendedEventSerializer for our models Participant, Club, Event and RecommendedEvent respectively. Before the serialization process, we checked some constraints. For instance, the participant should enter a valid BCC account,

the name, surname and email must be unique and the Bilkent ID must consist of 8 digits in order to register and use our application. In addition to this, we handled the other issues such as serializing an object in the OneToOneField or serializing multiple objects in the ManyToManyField. In this way, we could serialize all of the objects and send them to the respective endpoints without making an additional query in the client side.

4.2.3 views.py

In this module, we perform the CRUD operations on the serialized object. We can create a ModelViewSet class to automate the CRUD operations by specifying the queryset which is the set of query which is performed in any database table, the serializer class which converts the result of the query to the JSON formatted data, permission classes which controls who can access the serialized data in the API endpoint and lookup field which is a primary field of an object which will be queried and performed the CRUD operations on. We have written the serializer classes which are ParticipantViewSet, ClubViewSet, EventViewSet and RecommendedEventViewSet for our models Participant, Club, Event and RecommendedEvent respectively. In addition to this, we have created two APIView classes for the authentication purposes. One of them is called LoginView, which queries if the user can login successfully. It takes the Bilkent ID and password from the client side and first it queries the user-given Bilkent ID in the Participant table. If it finds the respective query, it compares the given password with the password in the database. If the validity is ensured then the server returns the object with Bilkent ID and password along with the HTTP 200 status code, otherwise the server side returns the HTTP 400 status code along with the error. The other APIView class is called RegisterView, which queries if the user can register successfully. It takes the Bilkent ID, first name, last name, email, pp and password from the client side and after checking the constraints specified in the models.py module it serializes the data. If the serialization is valid meaning the data has passed the check of all of the constraints, it posts the data to the register API endpoint and then to the Participant table in the database. If the POST operation is successful it returns the posted data along with the HTTP 201 status code, otherwise the server side returns the HTTP 400 status code along with the error.

4.2.4 urls.py

In this module, we generate the API endpoint urls for each of the ViewSets and APIViews so that the client side can perform the CRUD operations using these endpoints. We created an ‘auth/register’ endpoint for the registration operation in the RegisterView, ‘auth/login’ for the login operation in the LoginView, ‘viewset/..’ endpoint for the GET and POST operations for each Participant, Club, Event and RecommendedEvent object and ‘viewset/../<primary_key>’ endpoint for the UPDATE and DELETE operations for each Participant, Club, Event and RecommendedEvent object. We used a

DefaultRouter class to automate the URL routing and we registered the endpoints for each ViewSets to the router. The endpoints for each model are different. The client side can use these endpoints mentioned above with the specified headers, send a request to these API endpoints and receive a response in a straightforward manner.

4.2.5 settings.py

In this module, we can specify the settings of our application such as determining the allowed hosts, specifying installed apps and middlewares, specifying Django templates, the database engine, name, username, password and port, determining a path to store the static files such as HTTP, JavaScript and media files and password validators. To upload the backend functionality to Heroku, we adjusted the settings such as the choice of database PostgreSQL engine, using CORS headers application, adding the CORS headers middleware and WhiteNoise middleware and assigning a path for static files. Configuring the settings using the settings.py module helped us to upload the backend functionality to Heroku.

4.3 Database

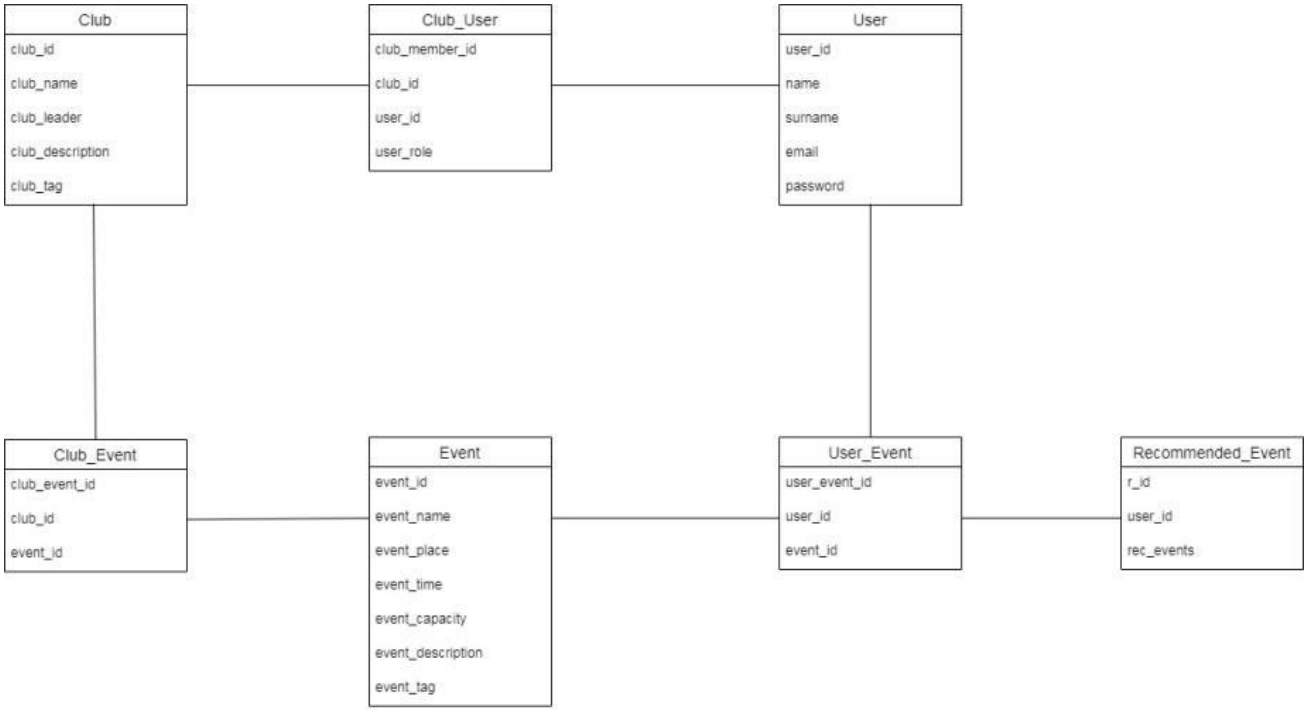

Figure 19. Entity Relationship Diagram

We first created an Entity Relationship Diagram for our database. In the diagram, we decided that the entities should be Participant, Club, Event and Recommended Event. The attributes of the tables are explained in detail in below.

4.3.1 Participant

- **bilkent_id:** Bilkent ID of a student or alumni. The primary key of the table. It is a positive integer field which has only 8 digits. The ordering is done with respect to the Bilkent ID.
- **first_name:** First Name of a student or alumni. It is a char field which has 50 character maximum length.
- **last_name:** Last Name of a student or alumni. It is a char field which has 50 character maximum length.
- **email:** BCC Email of a student or alumni. It must be a valid BCC domain account. It is a char field which has 50 character maximum length.
- **password:** The account password of a student or alumni. It is a char field which has 50 character maximum length.

4.3.2 Club

- **club_name:** Name of the Bilkent student club. It is a char field which has 50 character maximum length. The ordering is done with respect to the club name.
- **club_description:** The provided description of the Bilkent student club. It is a text field.
- **club_tags:** The tags of the Bilkent student club such as Science, Music, Programming and so on. It is a char field which has 200 character maximum length since it could have more than one tag.
- **logo:** The logo of the Bilkent student club. Default picture is given for those who need privacy. It is an image field which is stored in the images media folder created by Django.
- **leader:** The club leader of the Bilkent student club. Club and Participant entities have a one-to-one relationship which represents the leader of a specific club. We delete the leader with a cascade manner.
- **participants:** The participants of the Bilkent student club. Club and Participant entities have a many-to-many relationship which represents the members of a specific club.

4.3.3 Event

- **event_name:** Name of the organized event in Bilkent. It is a char field which has 50 character maximum length. The field must have a uniqueness constraint.
- **event_place:** Location of the organized event in Bilkent. It is a text field.
- **event_time:** The date and time of the organized event in Bilkent. It is a datetime field.

- **event_current_capacity:** The current capacity which represents how many participants are coming to the organized event in Bilkent. It is a positive integer field.
- **event_max_capacity:** The maximum capacity which represents how many participants can come to the organized event in Bilkent. It is a positive integer field.
- **event_tags:** The tags of organized events in Bilkent such as Science, Music, Programming and so on. It is a char field which has 200 character maximum length since it could have more than one tag.
- **event_points:** The GE 250/251 points of the organized event in Bilkent. It is a positive integer field. If other events which do not give the GE 250/251 points are specified then the value is 0.
- **event_zoom_link:** The zoom link of the online organized event in Bilkent. It is a char field which has 200 character maximum length. The field can be null if the event will not be held online.
- **participants:** The participants of the organized event in Bilkent. Event and Participant entities have a many-to-many relationship which represents the participants of a specific event. It can be null because of the probability that nobody participates in the specific event.
- **club:** The organizer club of the organized event in Bilkent. Event and Club entities have a many-to-one relationship which can be explained that many events can be organized by one club; however, one event cannot be organized by more than one club, the rarity of the phenomenon is quite high. It can be null because of the probability that the event could be the concert or other events which a student club does not involve in organizing. We delete the club with a cascade manner.

4.3.4 Recommended Event

- **event_name:** Name of the recommended event in Bilkent. It is a char field which has 50 character maximum length. The field must have a uniqueness constraint.
- **event_place:** Location of the recommended event in Bilkent. It is a text field.
- **event_time:** The date and time of the recommended event in Bilkent. It is a datetime field.
- event_star:
- **event_current_capacity:** The current capacity which represents how many participants are coming to the recommended event in Bilkent. It is a positive integer field.
- **event_max_capacity:** The maximum capacity which represents how many participants can come to the recommended event in Bilkent. It is a positive integer field.
- **event_tags:** The tags of recommended events in Bilkent such as Science, Music, Programming and so on. It is a char field which has 200 character maximum length since it could have more than one tag.

- **event_points:** The GE 250/251 points of the recommended event in Bilkent. It is a positive integer field. If other events which do not give the GE 250/251 points are specified then the value is 0.
- **event_zoom_link:** The zoom link of the online recommended event in Bilkent. It is a char field which has 200 character maximum length. The field can be null if the event will not be held online.
- **user:** The user who a recommended event will be recommended to in Bilkent. Event and Participant entities have a many-to-many relationship which can be explained that many events can be recommended to the participant and one specific event can be recommended to more than one participant. It can be null because of the probability that nobody participates in the specific recommended event.
- **club:** The organizer club of the recommended event in Bilkent. Recommended Event and Club entities have a many-to-one relationship which can be explained that many events can be organized by one club; however, one event cannot be organized by more than one club, the rarity of the phenomenon is quite high. It can be null because of the probability that the event could be the concert or other events which a student club does not involve in organizing. We delete the club with a cascade manner.

After creating our Entity-Relationship Diagram, we made a brainstorm on which database engine we should use. Finally, we decided to use PostgreSQL which is a Structured Query Language, because it is reliable, robust and it provides a great performance. It ensures the data integrity with primary/foreign keys with constraints, it optimizes and increases the performance by using Indexing, Advanced Indexing, Concurrency Control, JIT compilations of SQL expressions, it provides high security sense by using SCRAM-SHA-256 hashing algorithm, authentication protocols and it can be used with Python programing language and the data in the database can be exported in the JSON format. Besides, Heroku remote server provides a remote database whose database engine is PostgreSQL whose maintenance is taken care of by Heroku. After creating a remote database in Heroku and specifying the database on settings.py module, we uploaded the source code to Heroku Git and we managed to run the server without any problems.

4.4 Recommender Systems

To implement our model, we did an extensive research on how to implement a recommendation system. The conclusion of our extensive research has brought us the two ways of implementation: Matrix Generation with 2D Embedding and using Cosine Similarity Measurement or using Deep Learning. To use Deep Learning, we needed to have lots of data. However, the calculation of how

much data we can generate created a conclusion of not using the Deep Learning method. The maximum amount of data we can get is 50000 student x 80 events for a year; therefore, we concluded that our model would overfit especially in the beginning of the usage of application. Besides, we needed a well processing power and we thought about the integration of our model to the application as well; therefore, it needed to be efficient to satisfy the users' needs. As a result, we used the TF-IDF matrix as a type of 2D dataset and we used Cosine Similarity and SVD algorithms to generate fast and accurate results[1, 2, 3].

Our model uses Model Based Collaborative Filtering which is a generated matrix from user-item interactions where the model learns from the interactions matrix[1, 2, 3]. We used the Surprise library for processing our data[4, 5, 6]. Surprise library has a bunch of optimized built-in methods which can help us to optimize our ML model. We used the Pandas library to preprocess the data and we examined the ineffective data generated from the user itself. In the interaction matrix, we have only six values. We generated the value 0 for the users who didn't attend the event or didn't vote and we used a 1-5 rating scale to give the participants the opportunity to vote. The design of the model had a bunch of problems; however, we managed to tackle the problems one by one. The problems are mentioned below.

- Data Sparsity
- Cold Start Problem
- First Rater Problem
- Troller Rater Problem

4.3.1 Data Sparsity

Our interaction matrix is a very sparse matrix actually. The probability of the user not going to an event or not rating an event is hypothetically much higher because people are lazy. The matrix would consist of the higher amount of zeros. The fact that people are lazy has brought a conclusion of not using the Explicit Recommendation System. That is why, we didn't like the idea of going for the pure Content Based Method or Collaborative Filtering method. Instead, we used a hybrid model called Content-Boosted Collaborative Filtering[9]. The model actually uses both Content Based Method and Collaborative Filtering. However, the Collaborative Filtering method is a main method and the Content Based Method is the backbone of our main model. It solves the Data Sparsity problem by using the user rating as a main rating. If the user didn't prefer to give a rating to the event then it uses the outcome of the interaction matrix which is generated by the backbone Content Based Method. Our model Content-Boosted Collaborative Filtering solves the problem effectively by creating a pseudo-interaction matrix which is generated by both of the models[7].

4.3.2 Cold Start Problem

Cold Start Problem arises in three situations. The first one is when the new event is added to the interaction matrix. Since there is no or there are very few interactions, the pure Collaborative Filtering algorithm generates poor results because of the sparsity of the data or there are no start values which the Collaborative Filtering algorithm can based on. In addition to this, the Cold Start Problem creates another issue which is called popularity bias. When highly rated events are rated highly from the other participants; whereas, unpopular items are rated lowly by the other users. As a result, it creates an injustice and our recommendation algorithm would generate poor results when we use the pure methods. The second one is when the new participant is added to the interaction matrix. The newcomer students have no interaction with any event; therefore, the recommendations will be poor and randomized. The third one is when the interaction matrix is generated on the entirely new community. In this case, the interaction matrix will be 100% sparse and it will be impossible to recommend using the pure methods. Thus, the hybrid method helps us to generate the results with more quality. Our model Content-Boosted Collaborative Filtering solves the problem effectively by generating the pseudo-row for the participant and the pseudo-column for the event[8].

4.3.3 First Rater Problem

The First Rater problem can be summarized that the first rater of a newly added event can affect the recommendations deeply. When a participant rates a newly added event, since there is only a non-sparse data on the event axis which is the first rate, the recommendations are generated from this data segment. The first rate will dominate other rates, it will generate a popularity bias. As a result, the model will overfit and generate poor recommendations using only pure methods. Our model Content-Boosted Collaborative Filtering solves the problem effectively by not relying on the first rate data.

4.3.4 Troller Rater Problem

The definition of the Troller Rater Problem is to corrupt the interaction matrix by generating the random votes. As a result, our hybrid recommendation model cannot handle the issue of Troller Rater Problem by itself. To solve the problem, we implemented the Magic Barrier algorithm in our recommender system. We first use the RMSE metric for calculating the error between the results of the generated model and the actual user ratings. To minimize the error, we implement the Magic Barrier algorithm which is getting the variances of the user ratings, sum them up and average the sum by dividing the number of user ratings. Finally, we calculate the root of the result to find the Magic Barrier. To link the Magic Barrier and user coherence, we get the minus square of the Magic Barrier result[15]. We find the user inconsistencies by looking at how much variance the votes have. We

added this algorithm to our model to ensure the consistency of our recommendations and generate better results.

The model can be summarized with a flowchart which is shown below.

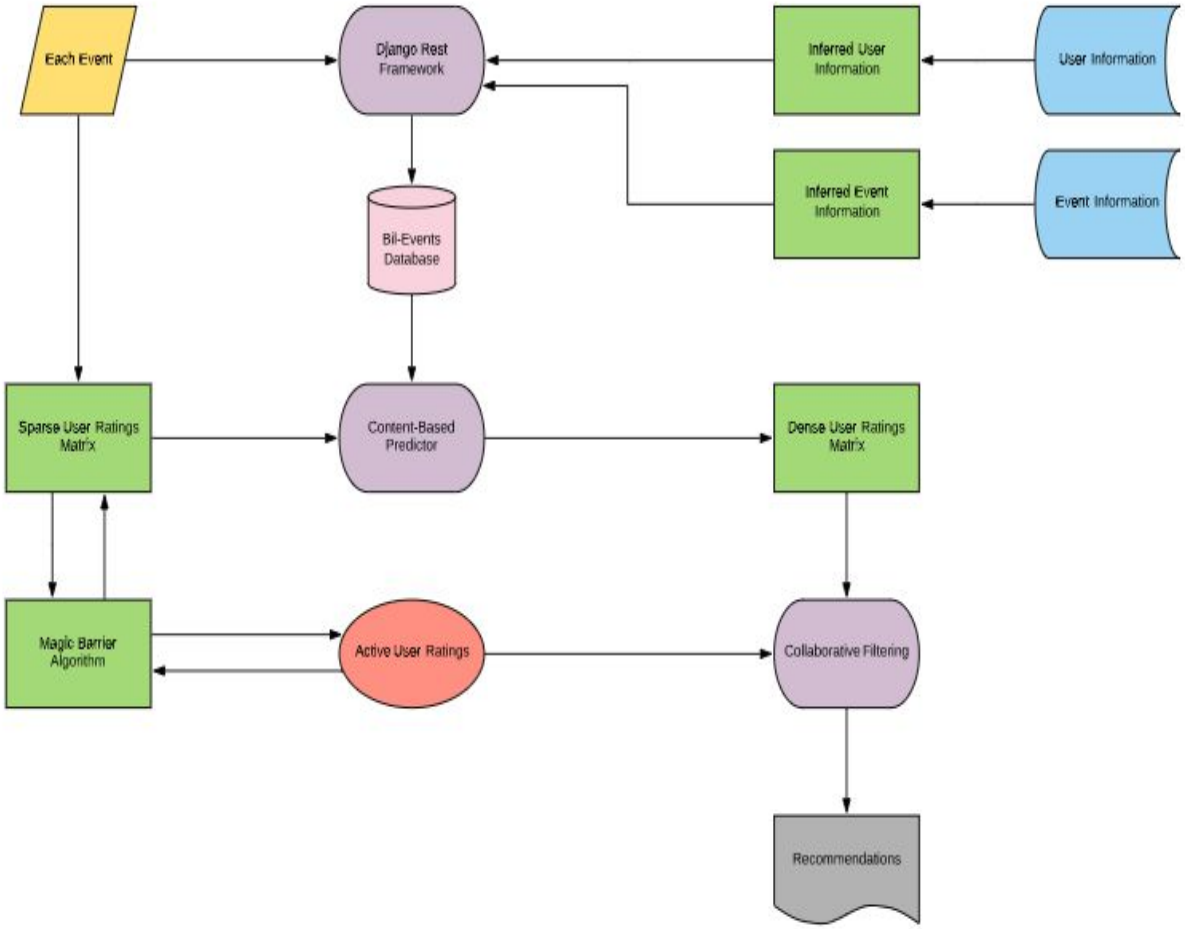

Figure 20. Recommender System Model

The explanation of the model will be as follows. First, we get the user and event information from our database using Django Rest Framework. Then, we generate inferred data from these raw data. For instance, we made the Data Mining from Participant model such as getting the entrance year from their ID and getting their department from their domain name of emails. For the events, we used event tags, current and maximum capacity of the events, GE 250/251 points of the events, the hosting clubs names of the events and even their location. We took the actual ratings to generate the sparse matrix. To check if there is any consistency in the sparse matrix, we used the Magic Barrier Algorithm mentioned above[15]. Then, we used the content based predictor to predict the events based on

inferred data we generated and older ratings the participants are provided. We implemented the content based recommendation system by using the Decision Tree algorithm and to boost the tree we used the XGBoost algorithm.[10] After implementation, we ran our pure content based predictor and generated the interaction matrix. We located the sparse data and exchanged the sparse data with the data we generated using the pure content based model and we didn't touch the data which has the actual user ratings, we named the result of interaction matrix as pseudo-interaction matrix. We used Harmonic Mean Rating to surpass the inaccuracies which is generated by few ratings and which yields misleadingly high correlations between the active user and other users. In addition to this, we used Self Rating to favor the active user and surpass the pseudo-active users effect on the interaction matrix. Finally, we generated the pseudo-interaction matrix by combining the Harmonic Mean Rating and Self Rating which are mentioned on the Content-Boosted Collaborative Filtering for Improved Recommendations paper[9]. After generating the dense interaction matrix, we fed the dense interaction matrix to our Collaborative Filtering algorithm. We used Matrix Factorization to get the eigenvalues and eigenvectors. We used these eigenspace values to generate the ranking of the most correlated events. We used the Singular Value Decomposition (SVD) algorithm for Matrix Factorization. Following this process, we have managed to implement a fully functional recommender system. Then, we generated a synthetic interaction to see if the system is working [4, 5, 6].

After ensuring that the system is fully functional, we generated top recommended events for the specific user from the Recommendation System and we stored them in the Recommended Events table. We sent the data to the client side and we managed to implement the Recommended Event functionality.

5. Testing Details

Checking the application's functionality is a long process if we check manually every time. Also test automations can find more bugs that we have not faced yet while manually testing . Therefore the tests are runned by using Appium Test Automation since Bil-Events is an android application. In the test, Java programming language is used. All tests are done as closed.

In the closed Alpha test, there were a lot of bugs. In the frontend, bugs were generally caused by either incorrect length of inputs or incorrect format of the input. For instance, in the register screen, we realized that there is no character limit for username, name , surname ,email and password. Moreover, in the login screen, when the user logins with special characters in school ID like "*", "_" and etc. , app crashes. Also there was no character limit for ID. So this was another problem that caused the app to crash because eight characters are supposed to be entered as ID. In the backend, there were some bugs about the user register since the user was not authorized for adding data to the

database by registering. Furthermore, post requests could cause problems due to lack of checking uniqueness of the datas. In the backend, we tested the models using the test module provided by Django. The test module is implemented using the unittest module in Python programming language. We saw that there were a lot of bugs caused by Django Rest Framework itself such as the problems in database models, serialization, authentication, putting the backend functionality to Heroku and Recommender Systems. In the database models, the problems emerged from deciding which attributes should be in the models, managing model relationships and general design issues. In the serialization, the problems emerged from failing to serialize the data and serialization of multiple model objects in the JSON format. In the authentication, since we didn't use the User model provided by Django because of designing a Custom user model, we needed to implement and test the registration and login systems by ourselves. Finally, we managed to implement the systems without problems. When uploading the backend functionality to Heroku, we tried to upload all of the frontend and backend functionalities to the server; however, it turned out that Heroku does not support React Native applications. Instead, we separated the frontend functionality from the backend functionality and we uploaded the backend functionality to the remote server. After that, we made a bunch of tests to see if the backend functionality is working on the remote server. In the implementation of a simple Recommender System, in order to handle the optimization issues, correct implementation of algorithms and data fetch from our database, we made tests to see if it is working smoothly.

After the alpha test, we believed that we fixed the bugs and made Bil-Events a bug free application. If we publish the app, the open beta test will be carried out to check more detaily.

6. Maintenance Plan and Details

Since we use an online database, Heroku, for the app, we use a free version of database and the free database version of Heroku provides us a limited database such as low storage around 512 MB, high latency, no restoration opportunity and fully managed backup.That database will not be enough for a very large number of users right now. Therefore, if the number of users expands, we may need to change our database to satisfy user requests and make the app better. Other than that this app is only available for android users now. For the future we may add an iOS option and web interface according to user demands. We cannot ignore the number of iOS users in the Bilkent that want to use our app. We will keep the database updated and increase efficiency as the number of users increases.

7. Other Project Elements

7.1 Consideration of Various Factors in Engineering Design

7.1.1 Adaptation to Bilkent University and Other Universities

Our application can reach other universities too. After having published, other universities may want our app to organize their events. This large scale creates the problem of storage. It affects both maintainability and sustainability.

7.1.1.1 Safety

This constraint is very important. Since Bilkent University does not provide us with their database, we have to create our own. Someone from not Bilkent but another university and from Bilkent but someone who uses another Bilkent student's ID can create an account in the app because there is no opportunity to check this account. So safety is one of the top important things according to our criteria. If a contract is signed with Bilkent, privacy of Bilkent University's Database will be secured. However, in this condition, correctness of the datas cannot be interrogated. Therefore, if a university wants that event organizer app, there should be an access to the university's database and the app will serve better.

7.1.1.2. Economic

Adaptation to data will require money for storage. It becomes harder to control something that is growing because any new additional feature will be needed for maintenance. While enhancing the project, money is required since what we want is sold to enhance the project like a more efficient online database. However, if our users grow exponentially we will make investment for data storage and security of the data. Furthermore, in order to add the app to the Apple Store, we need economic support.

7.1.1.3. Global

If only one server of our application serves the users, the efficiency will dramatically fall as new users and clubs are added. Getting the data from the server will be problematic if the people from other universities try to use our developed application, this problem is getting bigger and bigger. For instance, except Bilkent University, METU may want our application too. When this happens, we have to have another server for METU only. Since our app is not global right now, it is not causing a problem, but when it's getting bigger we will increase efficiency with updates.

Judgement Description:	Adaptation to Bilkent University and Other Universities	
	Impact Level (out of 10)	Impact Description
Impact in Safety	10	Safety becomes harder to establish since the university does not share the database with us.
Impact in Economic	6	Purchasing new data storage units can be costly.
Impact in Global	7	Spreading to other universities will make sustainability harder.

Figure 21. Judgement Table

7.1.2 Adapting to Pandemic

We started our project before the global pandemic started. At first we didn't think it would be this long. It will affect the event schedule and our application needs to be adapted to this environment.

7.1.2.1 Safety

Since the user can join online events, human-human interaction will be prevented. Also, users will be informed about all events compared to now. While people are getting socialized, they won't get Covid-19 virus since there is no interaction between people.

7.1.2.2 Social

Pandemic also affected our daily life and education. Transition to normal to this current situation was difficult. Therefore, We decided to add a zoom link button for each event. Since some clubs are still active through online platforms like zoom. We thought that it would be a good feature and related to the current situation of the world. As a result, the activeness of the clubs won't be affected by the pandemic too much and students and staff will benefit from that upgrade. Also clubs are encouraged to continue meetings and activities.

Judgement Description:	Adapting to Pandemic	
	Impact Level (out of 10)	Impact Description
Impact in Safety	8	People's safety is significant in pandemic.
Impact in Social	8	People can join online events in pandemic.

Figure 22. Judgement Table 2

7.2 Ethics and Professional Responsibilities

7.2.1 Ethical Issues

- Bil-Events application does not collect personal information such as contact number, credit card information, TC identification number. In the case of storing other personal data, we will encrypt the data and make sure that the data is secure.
- If Bilkent University provides us with their database, using that database for our purposes is not ethical. Therefore, if the database is provided, there will be an agreement between creators of Bil-Events and Bilkent University in order to give Bilkent University a legal right about protecting its own data.

7.2.2 Professional Issues

- Source code of the application is stored as a private GitHub code.
- Each member is responsible for helping other members in a place they lack knowledge. In this way both sides learn how to work collaboratively.
- Each member is responsible for making extensive research about which technology they use and whether the technology that the member uses is compatible with the technologies which the remaining members use.
- Each member is responsible for keeping in touch with the supervisor, for making a respectable amount of improvements each week and for sharing the improvements with the supervisor.
- Each member is responsible for getting advice from the supervisor if the other members have

the lack of knowledge about the problem that a specific member is having.

- We meet up with each other at least once a week. These meetings were face-to-face meetings or on a digital platform such as “Skype” or “Discord”. We keep in touch with our supervisor regularly, if he is available.

7.3 Judgements and Impacts to Various Contexts

Bil-Events’ purpose is to increase socialness in the university, decrease paper consumption caused by posters and inform everyone about events. For maintenance of the app, people should sign up to the app and both join and add events to the app. When the club leaders notice that the number of newcomers is increasing, our app will spread. Since online meetings are popular these days, we made a judgement that people will want to join events both online and physically. We have to think that people will want to be informed about the events. Bil-Events was constructed on this main idea. To make it happen we tried to make a user-friendly and safe app.

Judgement Description:	People will want to be informed about the events and be more social	
	Impact Level	Impact Description
Impact in Global Context	High	Since Bil-Events will be open for whole university, every user should want to join an event or every club leader should want to add an event.
Impact in Economic Context	Low	Users will not spend any money for using our app. However, we should make investment in our app if user numbers are increased.
Impact in Environmental Context	Medium	Our app should be considered as a safe and usable app in the environment.
Impact in Societal Context	High	Since, Bil-Events is a social app, it should make a good impact on society.Users should suggest our app to others in order to help us grow.

Figure 23. Judgement Table

7.4 Teamwork Details

7.4.1 Contributing and functioning effectively on the team

Osman Can Yıldız :

- Research about React-Native Framework, JavaScript and Expo
- Frontend Design(Login, Register, Forgot Password, Upcoming Events, Selected Events, Club Lists, Recommended Events, Past Events, Add Event)
- Frontend - Backend connection
- UI/UX Designs
- All reports
- Testing with Appium Test Automation (Java)
- User Manual - Guide

Can Savcı:

- Research about Heroku API and uploading the backend to the server
- Search for remote database and server
- PostgreSQL and its Usage
- Research about Django and Django Rest Framework with Python
- Backend Design
- Backend Test using test Module
- Frontend - Backend connection
- All reports
- Implementation of a Simple Recommender System

Kasım Sarp Ataş:

- Some reports

7.4.2 Helping Creating A Collaborative and Inclusive Environment

Each member is responsible for helping other members in a place they lack knowledge. In this way both sides learn how to work collaboratively. Each member is responsible for making extensive research about which technology they use and whether the technology that the member uses is compatible with the technologies which the remaining members use.

7.4.3 Taking Lead Role and Sharing Leadership On The Team

We did not specifically assign a leader for this project. But we know that leadership is important because it sets a clear vision and communicates effectively to subordinates/colleagues. The clear vision gives us a better understanding of organisational direction and makes them realise their roles and responsibilities.

7.4.4 Meeting Objectives

In the meetings, our objective was to brainstorm the design issues in the frontend, backend, database and recommender system and combine the functionalities of the app. We gave utmost importance to the design so that we could implement the application quickly and without any objections of any group members. We managed to meet often to debug the code, to write the reports and to brainstorm Requirements Elicitation, Analysis, System and Object Design and which Design Pattern we should use when implementing the project. In addition to this, we conducted the research of which platforms we should use for the project and we meet and brainstorm the availability of the platforms as well. We took the feedback of our supervisor Prof. Dr. Halil Altay Güvenir and our jury members Prof. Dr. Fazlı Can and Prof. Dr. Özgür Ulusoy on the reports very seriously and we meet up to discuss the feedback and how we can improve our application.

7.5 New Knowledge Acquired and Applied

New knowledge that we have learned while we are designing the project is the main goal of this project. For this purpose, we use an online learning strategy which we already discussed and found online courses. For the others, we will use web search methods, articles and analyze other applications. The new knowledges that we are learning to make an efficient, trustworthy and reliable application are given in the list below:

- Android Programming with React Native Framework
- Recommendation Engine
- Visual Studio Code
- Heroku
- Django Rest Framework
- Database encryption
- Server handling
- Photoshop CC for design purposes

7.5.1 Android Programming with React Native Framework

We found an Android Studio course from Udemy in order to learn both the basic and advanced structure of Android programming. For now, we are practicing while watching the course. After having finished the course, we will start actual implementation of Bil-Events.

7.5.2 Recommendation Engine

We have made a simple Recommended System to manage the recommendations taking the cost of the running time of the application into consideration. The recommendation engine is the one of crucial

parts of our project since we don't want Bil-Events to recommend users irrelevant events that users won't interest. Therefore, we learned all steps of the recommendation engine such as collaborative filtering and its types, data needed to build a recommender, libraries available to build recommenders, use cases and challenges of collaborative filtering. After that, we finished implementing the Recommender Engine System.

7.5.3 Visual Studio Code

In the implementation phase, we used Visual Studio Code IDE because it was the best IDE for our purposes. It is a light-weight IDE which is easy to use, manage and customize. It supports both React.js and Python programming language; therefore, Visual Studio Code made it easy to develop our application since we used the same IDE. In this way, we could manage the frontend and backend connection without any problems. Visual Studio Code is an open-source project which runs on every machine. It has features such as autocomplete and syntax highlighting via IntelliSense. Debugging is easy with Visual Studio Code. It supports Git as well so that we could upload our source code to the private repository in Github and Heroku Git. Deployment is easy with Visual Studio Code as well since it supports Git and Heroku CLI, we could deploy our application to Heroku.

7.5.4 Heroku

We needed a remote server to handle the production phase smoothly and provide the users 24/7 connection to our application. That is why research about remote servers is conducted. We decided to use Heroku, because we calculated the worst case memory storage of our database and based on our evaluations, Heroku can be used and it can run without any problems. Besides, Heroku is free to use and the maintenance is easy when using Heroku. The platform is flexible, easy to use and there is no need to maintain the server, hardware and infrastructure. Instead of connecting via local server or giving a bunch of money to AWS or Google Cloud, Heroku protects its advantages among other PaaSes.

7.5.5 Django Rest Framework

Django Rest Framework is a powerful tool to serialize the database using JSON. Since React Native is a framework for building native apps using React.js, we needed to use a framework which has a JSON serialization mechanism so that we can make CRUD operations on the data and fetch the data to our React Native applications. It is easy to use, the documentation of Django Rest Framework is easy to understand, it supports every functionality in Django as well. It provides a Web browsable API, DRF makes it easy to debug and manage the connection of the frontend and backend of the application.

DRF has a lot of utilities such as permissions which manages the authentication process, routers which makes us create the API endpoints of our application.

7.5.6 Server Handling

Since we are using Heroku for the server, we handled the maintenance issues using a PaaS utility. We ran our frontend server using Expo and we handled the start time and the end time of the server. In addition to this, since Expo is not stable enough for the cache issues, we needed to maintain the server by clearing the cache and restarting the server. For the maintenance of the backend server on Heroku, we used some techniques such as starting and ending the server, resetting the database in the server, handling the migrations or recent changes of our backend functionality via Heroku Bash and managing the PostgreSQL of the server.

7.5.7 Adobe Photoshop CC

Adobe Photoshop CC will be used for design purposes. We used it for our logo,mock ups and icon design. For further background, images and icons that are required in the application will be done by using Photoshop CC.

8. Conclusion and Future Work

We managed to implement the fully functional Bil-Events application by following the standards of the software engineering, implementation guidelines, documentations of the technologies we used, adopting collaborative mindset, dedication and hardwork. We hope that Bilkent University students and alumni can benefit from our application in the best way possible. Based on the feedback of the end users and our innovative mindset, we can improve the application and add extra user-friendly features. For instance, we can add the Google Maps API to locate the event location and share with the participants who want to go to that location or we can implement the message system which can help the participants to communicate with each other. However, overall it fulfills the visions and missions of our objective and it provides these solutions aesthetically.

9. Glossary

JSON : JavaScript Object Notation

CLI: Command line interface

API: Application programming interface

PaaS: Platform as a service

DRF : Django rest framework

AWS: Amazon Web Services

UI: User interface

UX: User experience

METU: Middle East Technical University

TC: Türkiye Cumhuriyeti

BAIS: Bilkent Administrative Information System

YÖK: Yükseköğretim Kurulu

CRUD: Create-Read-Update-Delete

10. References

[1] A. Ng, *Youtube.com*, 2020. [Online]. Available: https://www.youtube.com/watch?v=9siFuMMHNIA. [Accessed: 27- Dec- 2020].

[2] "5 steps to setting up a recommender system", *Klipfolio.com*, 2020. [Online]. Available: https://www.klipfolio.com/blog/recommender-system. [Accessed: 27- Dec- 2020].

[3] "Introduction to recommender systems", *Medium*, 2020. [Online]. Available: https://towardsdatascience.com/introduction-to-recommender-systems-6c66cf15ada. [Accessed: 27- Dec- 2020].

[4] "Getting Started — Surprise 1 documentation", *Surprise.readthedocs.io*, 2020. [Online]. Available: https://surprise.readthedocs.io/en/stable/getting_started.html. [Accessed: 27- Dec- 2020].

[5] "Building and Testing Recommender Systems With Surprise, Step-By-Step", *Medium*, 2020. [Online]. Available: https://towardsdatascience.com/building-and-testing-recommender-systems-with-surprise-step-by-step-d4ba702ef80b. [Accessed: 27- Dec- 2020].

[6] "dataset module — Surprise 1 documentation", Surprise.readthedocs.io, 2020. [Online]. Available: https://surprise.readthedocs.io/en/stable/dataset.html#dataset. [Accessed: 27- Dec- 2020].

[7] "How to improve recommendations for highly sparse datasets using Hybrid Recommender Systems?", *Medium*, 2020. [Online]. Available: https://medium.com/ai-in-plain-english/how-to-improve-recommendations-for-highly-sparse-datasets-using-hybrid-recommender-systems-1a4366e65cff. [Accessed: 27- Dec- 2020].

[8] "Cold start (recommender systems)", *En.wikipedia.org*, 2020. [Online]. Available: https://en.wikipedia.org/wiki/Cold_start_(recommender_systems)#:~:text=The%20item%20cold-start%20problem,item%27s%20interactions%20to%20make%20recommendations. [Accessed: 27- Dec- 2020].

[9] 2020. [Online]. Available: https://www.cs.utexas.edu/~ml/papers/cbcf-aaai-02.pdf. [Accessed: 27- Dec- 2020].

[10] *Cs.fit.edu*, 2020. [Online]. Available: https://cs.fit.edu/~pkc/apweb/related/pazzani07aw.pdf. [Accessed: 27- Dec- 2020].

[11] "Data Preprocessing: 6 Necessary Steps for Data Scientists | Hacker Noon", *Hackernoon.com*, 2020. [Online]. Available: https://hackernoon.com/what-steps-should-one-take-while-doing-data-preprocessing-502c993e1caa. [Accessed: 27- Dec- 2020].

[12] "Hypertext Transfer Protocol", *En.wikipedia.org*, 2020. [Online]. Available: https://en.wikipedia.org/wiki/Hypertext_Transfer_Protocol. [Accessed: 27- Dec- 2020].

[13] "PostgreSQL", *En.wikipedia.org*, 2020. [Online]. Available: https://en.wikipedia.org/wiki/PostgreSQL. [Accessed: 27- Dec- 2020].

[14] "Bilkent Universitesi - Ogrenci Dekanligi - Ogrenci Konseyi Koordinatorlugu", *Ogrencidekanligi.bilkent.edu.tr*, 2020. [Online]. Available: http://www.ogrencidekanligi.bilkent.edu.tr/okk/index.html. [Accessed: 27- Dec- 2020].

[15] A. Said and A. Bellogín, "Coherence and inconsistencies in rating behavior: estimating the magic barrier of recommender systems", 2020. [Online]. Available: https://link.springer.com/article/10.1007/s11257-018-9202-0#Sec11. [Accessed: 27- Dec- 2020].

[16] "Running Tests," *Appium*. [Online]. Available: http://appium.io/docs/en/writing-running-appium/running-tests/. [Accessed: 27-Dec-2020]

[17] "Core Components and APIs · React Native", *Reactnative.dev*, 2020. [Online]. Available: https://reactnative.dev/docs/components-and-apis. [Accessed: 27- Dec- 2020]

[18] "BuilderX", *Builderx.io*, 2020. [Online]. Available: https://builderx.io/. [Accessed: 27- Dec- 2020]